Minimum Set Cover Project

Group Description and Provisional Planning

Advanced Algorithms – Master DSC/MLDM

November 4 – December 15, 2026

1. Group Members

No.	Full Name	Student ID	Program
1	NGUYEN Phuc-Minh	27008568	MLDM
2	CAO Phuong-Dang	27010835	MLDM
3	PHAM Tuan-Hiep	27010841	MLDM
4	VO Nguyen-Phuong-Quynh	27012169	MLDM

2. Project Objectives and Main Tasks

The objective of this project is to implement and experimentally evaluate different algorithms for solving the Minimum Set Cover Problem (MSCP), a classical NP-hard combinatorial optimization problem [1].

The main tasks include:

- **Problem Modeling:** Define the problem, establish a unified data representation, and specify common input/output formats for all algorithms.
- **Random Instance Generation:** Develop a generator that creates Set Cover instances with configurable universe sizes, numbers of candidate subsets, and instance densities.
- **Algorithm Implementation:** Implement Brute Force, Branch and Bound, Greedy, Dynamic Programming, and additional methods.
- **Correctness Testing:** Verify the correctness of each implementation using small instances and compare exact algorithm results.
- **Benchmark Evaluation:** Evaluate the algorithms on artificially generated instances and existing unweighted benchmark instances.
- **Performance Comparison:** Compare running time and solution quality in terms of solution cardinality and optimality gap.
- **Reporting and Documentation:** Document the algorithms, experimental setup, results, analysis, and implementation details.

3. Task Distribution and Individual Contributions

Member	Assigned Tasks	Contribution
Dang	Implementation of the Brute-force algorithm.	10%
	Development of the Random Instance Generator.	8%
	Testing of the Dynamic Programming and Other algorithms.	7%
Hiep	Implementation of the Branch and Bound algorithm.	10%
	Implementation of an additional solution method.	8%
	Testing of the Greedy and Heuristic algorithms.	7%
Minh	Implementation of the Greedy algorithm.	10%
	Implementation of a heuristic/metaheuristic method.	8%
	Testing of the Branch and Bound and Brute-force algorithms.	7%
Quynh	Problem modeling and definition of input/output formats.	7%
	Implementation of the Dynamic Programming algorithm.	10%
	Experimental comparison and result analysis.	8%
Total		**100%**

4. Provisional Timeline and Milestones

Milestone	Tasks	Responsible Members
Project Planning (Nov 4 – Nov 8)	Finalize group organization and task allocation.	All members
	Define the Set Cover problem representation and common input/output format.	Quynh
	Implement the initial Random Instance Generator.	Dang
Core Algorithm Implementation (Nov 9 – Nov 15)	Implement the Brute-force algorithm.	Dang
	Implement the Branch and Bound algorithm.	Hiep
	Implement the Greedy algorithm.	Minh
	Implement the Dynamic Programming algorithm.	Quynh
Additional Methods and Integration (Nov 16 – Nov 24)	Complete and debug the main algorithm implementations.	All members
	Implement an additional solution method and a heuristic/metaheuristic method.	Hiep, Minh
	Improve the random generator if necessary and integrate all algorithms into a unified codebase.	Dang, All members
	Standardize interfaces, perform correctness testing, and prepare benchmark instances.	All members
Experimental Evaluation (Nov 25 – Dec 5)	Conduct experiments on generated instances with different sizes and densities.	All members
	Evaluate algorithms on existing unweighted benchmark instances.	All members

Milestone	Tasks	Responsible Members
	Collect running time and solution quality measurements.	All members
	Compare results with known optimal or best-known solutions.	Quynh, All members
Result Analysis and Documentation (Dec 6 – Dec 12)	Analyze and visualize experimental results.	Quynh
	Identify strengths and weaknesses of each algorithm under different instance characteristics.	All members
	Write the project report, including algorithm descriptions, experimental setup, results, and discussion.	All members
	Prepare installation instructions, usage documentation, and the required checklist.	All members
Final Review and Submission (Dec 13 – Dec 15)	Proofread the report, perform final code testing, and verify reproducibility.	All members
	Organize and submit the final project archive.	All members

5. Testing and Evaluation Procedures

The group plans to use the following procedures to ensure correctness and evaluate algorithm performance.

5.1 Correctness Testing

- Verify that every returned solution covers all elements in the universe.
- Check that the reported cardinality matches the number of selected subsets.
- Compare the results of exact algorithms on small instances to verify correctness.
- Test edge cases, including small instances and instances with different structural characteristics.

5.2 Artificially Generated Instances

- Use the Random Instance Generator to create instances with different universe sizes, numbers of candidate subsets, and densities.
- Evaluate how changes in problem size and instance structure affect algorithm performance.
- Use consistent generated instances when comparing different algorithms.

5.3 Benchmark Evaluation

- Use existing unweighted Set Cover benchmark instances from the USCP_instances repository [2].
- Evaluate the algorithms on instances of different sizes and structural characteristics.
- Compare solution quality with known optimal or best-known values whenever available.

5.4 Performance Evaluation

The evaluation will focus on two main aspects:

- **Running Time:** Measure and compare the execution time of each algorithm.
- **Solution Quality:** Compare the cardinality of the resulting covers and calculate the optimality gap when a reference solution is available.

For randomized methods, experiments will be repeated when necessary to evaluate the stability of the results. Numerical results, graphs, and error bars will be provided where appropriate.

5.5 Result Analysis

- Compare exact algorithms and heuristic approaches in terms of efficiency and solution quality.
- Analyze the impact of instance size and density on algorithm performance.
- Identify the types of instances on which each algorithm performs well or poorly.
- Discuss the strengths, limitations, and practical applicability of each method.

6. Expected Deliverables

By the final submission deadline, our group expects to complete:

1. A unified implementation of all selected algorithms.
2. A functional Random Instance Generator.
3. Correctness testing and debugging results.
4. Experimental results on artificially generated instances and existing benchmarks.
5. Comparative tables, graphs, and performance analysis.
6. A complete project report in PDF format.
7. Source code with installation, execution, and documentation instructions.
8. A completed project checklist and a record of the actual task distribution and planning.

This is a provisional plan. Actual progress, changes in task allocation, and deviations from the proposed schedule will be documented and discussed in the final report.

References

[1] R. M. Karp, "Reducibility Among Combinatorial Problems," in *Complexity of Computer Computations*, pp. 85–103, 1972. doi: 10.1007/978-1-4684-2001-2_9.

[2] P. Innamorati, "USCP_instances: Unicost Set Cover Problem Instances," GitHub repository. Available: `https://github.com/pinam45/USCP_instances`. Accessed: November 4, 2026.